



Σχεδιασμός Ενσωματωμένων Συστημάτων
9ο Εξάμηνο ΗΜΜΥ

6η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ

Cross-compiling προγραμμάτων για ARM αρχιτεκτονική

Ομάδα 20	
Σπυρίδων Αγάθος	03120858
Παναγιώτης Κατσαλίφης	03118939

Ο κώδικας των ασκήσεων έχει αναρτηθεί στο [Github](#).

Ασκηση 1

1. Οι armel και armhf είναι δύο εκδόσεις της ARM αρχιτεκτονικής, οι οποίες χειρίζονται διαφορετικά τις πράξεις κινητής υποδιαστολής. Η armel είναι πιο παλιά κι εκτελεί αυτές τις εντολές με software emulation, ενώ η armhf παρέχει φυσική μονάδα για FP operations.
2. Χρησιμοποιήσαμε την arm-cortexa9_neon-linux-gnueabihf, επειδή αυτή είναι συμβατή με το λειτουργικό και τον εικονικό επεξεργαστή που ορίσαμε στο QEMU. Εάν χρησιμοποιούσαμε ένα πιο generic toolchain, όπως π.χ. το arm-unknown-linux-gnueabi το παραγόμενο εκτελέσιμο ενδέχεται να ήταν λιγότερο efficient, ενώ εάν χρησιμοποιούσαμε κάποια τελείως διαφορετική αρχιτεκτονική, όπως π.χ. την mips64el-multilib-linux-uclibc δεν θα μπορούσαμε να τρέξουμε το εκτελέσιμο καθώς δεν θα ήταν συμβατό με την αρχιτεκτονική του VM.
3. Χρησιμοποιήσαμε την glibc, και συγκεκριμένα το version 2.19, επειδή το VM μας χρησιμοποιεί αυτήν την έκδοση.

4. Στην αρχή δοκιμάσαμε να χρησιμοποιήσουμε την glibc version 2.34 (επειδή αυτή ήταν η τελευταία διαθέσιμη έκδοση που παρείχε το crosstool). Το cross compilation ολοκληρώθηκε επιτυχώς, αλλά το εκτελέσιμο δεν έτρεχε λόγω αυτού του incompatibility. Έπειτα ρίξαμε όλα τα versions (του πυρήνα, των binutils, του gcc και της glibc) ώστε να ταιριάζουν με αυτά του target μηχανήματος, όμως αντιμετωπίσαμε διάφορα errors κατά το build και δεν καταφέραμε να κάνουμε cross-compile.

Παρόλαυτά, το output των εντολών file και readelf είναι ως εξής:

```
file phods_crosstool.out
```

```
phods_crosstool.out: ELF 32-bit LSB executable, ARM, EABI5
version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-
armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not stripped
```

```
readelf -h -A phods_crosstool.out
```

ELF Header:

```
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF32
  Data:                               2's complement, little
  endian
  Version:                            1 (current)
  OS/ABI:                             UNIX - System V
  ABI Version:                        0
  Type:                               EXEC (Executable file)
  Machine:                            ARM
  Version:                            0x1
  Entry point address:                 0x104c0
  Start of program headers:           52 (bytes into file)
  Start of section headers:           13732 (bytes into file)
  Flags:                              0x5000400, Version5 EABI,
  hard-float ABI
  Size of this header:                 52 (bytes)
```

Size of program headers: 32 (bytes)
Number of program headers: 9
Size of section headers: 40 (bytes)
Number of section headers: 36
Section header string table index: 35

Attribute Section: aeabi

File Attributes

Tag_CPU_name: "7-A"
Tag_CPU_arch: v7
Tag_CPU_arch_profile: Application
Tag_ARM_ISA_use: Yes
Tag_THUMB_ISA_use: Thumb-2
Tag_FP_arch: VFPv3
Tag_Advanced_SIMD_arch: NEONv1
Tag_ABI_PCS_wchar_t: 4
Tag_ABI_FP_rounding: Needed
Tag_ABI_FP_denormal: Needed
Tag_ABI_FP_exceptions: Needed
Tag_ABI_FP_number_model: IEEE 754
Tag_ABI_align_needed: 8-byte
Tag_ABI_align_preserved: 8-byte, except leaf SP
Tag_ABI_enum_size: int
Tag_ABI_VFP_args: VFP registers
Tag_CPU_unaligned_access: v6
Tag_MPextension_use: Allowed
Tag_Virtualization_use: TrustZone

5. Χρησιμοποιώντας τον έτοιμο cross compiler της linaro, συγκρίνουμε το μέγεθος του εκτελέσιμου με αυτό του crosstool:

```
root@debian-armhf ~# du -b phods_linaro.out
8888 phods_linaro.out
root@debian-armhf ~# du -b phods_crosstool.out
15172 phods_crosstool.out
```

Η διαφορά αυτή πιθανώς οφείλεται στο ότι ο cross compiler της linaro θα κάνει πιο πολλά optimizations ως προς το μέγεθος του παραγόμενου αρχείου, σε σχέση με τον configuration που έχουμε στο ct-ng.

Περισσότερες πληροφορίες για το εκτελέσιμο παίρνουμε μέσω των εντολών file και readelf:

```
file phods_linaro.out
```

```
phods_linaro.out: ELF 32-bit LSB executable, ARM, EABI5 version
1 (SYSV), dynamically linked, interpreter /lib/ld-linux-
armhf.so.3, for GNU/Linux 3.1.1,
BuildID[sha1]=38f1390dc6b5eca2d0ccad5e2a20984ddae70fb, not
stripped
```

ELF Header:

```

Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
Class:                               ELF32
Data:                                 2's complement, little
endian
Version:                             1 (current)
OS/ABI:                               UNIX - System V
ABI Version:                          0
Type:                                 EXEC (Executable file)
Machine:                              ARM
Version:                              0x1
Entry point address:                  0x8479
Start of program headers:             52 (bytes into file)
Start of section headers:             4264 (bytes into file)
```

Flags: 0x5000402, Version5 EABI,
hard-float ABI, <unknown>

Size of this header: 52 (bytes)

Size of program headers: 32 (bytes)

Number of program headers: 8

Size of section headers: 40 (bytes)

Number of section headers: 31

Section header string table index: 28

Attribute Section: aeabi

File Attributes

Tag_CPU_name: "7-A"

Tag_CPU_arch: v7

Tag_CPU_arch_profile: Application

Tag_ARM_ISA_use: Yes

Tag_THUMB_ISA_use: Thumb-2

Tag_FP_arch: VFPv3-D16

Tag_ABI_PCS_wchar_t: 4

Tag_ABI_FP_rounding: Needed

Tag_ABI_FP_denormal: Needed

Tag_ABI_FP_exceptions: Needed

Tag_ABI_FP_number_model: IEEE 754

Tag_ABI_align_needed: 8-byte

Tag_ABI_align_preserved: 8-byte, except leaf SP

Tag_ABI_enum_size: int

Tag_ABI_HardFP_use: Deprecated

Tag_ABI_VFP_args: VFP registers

Tag_CPU_unaligned_access: v6

6. Τρέχοντας το εκτελέσιμο που παράχθηκε από το crosstool παίρνουμε το error:

```
./phods_crosstool.out: /lib/arm-linux-gnueabi/libc.so.6: version
`GLIBC_2.34' not found (required by ./phods_crosstool.out)
```

Δηλαδή, το εκτελέσιμο έχει για dependency την έκδοση 2.34 της glibc και έτσι είναι αδύνατο να εκτελεστεί καθώς το σύστημα έχει glibc 2.19 (ldd -version)

7. Χρησιμοποιώντας το -static flag ο cross compiler φροντίζει όλα τα δυναμικά dependencies του εκτελέσιμου να ενταχθούν σε αυτό, το οποίο αυξάνει κατά πολύ το μέγεθός του, αλλά ως αποτέλεσμα μπορεί να εκτελεσθεί ανεξαρτήτως από το εάν το target διαθέτει τα αντίστοιχα dependencies (όπως την glibc, binutils κλπ.) ή όχι. Πράγματι, το μέγεθος των εκτελέσιμων είναι τώρα πολύ μεγαλύτερο:

```
root@debian-armhf ~# du -sh phods_linaro_static.out
500K phods_linaro_static.out
root@debian-armhf ~# du -sh phods_crosstool_static.out
2.6M phods_crosstool_static.out
```

Όμως και οι δύο cross-compilers παράγουν τώρα εκτελέσιμα που μπορούν να τρέξουν στο target μας.

8.

A) Το πρόγραμμα δεν μπορεί να εκτελεστεί λόγω διαφορετικής αρχιτεκτονικής (έχει παραχθεί για arm αρχιτεκτονική ενώ το σύστημά μου είναι x86).

B) Εάν και το εκτελέσιμο γνωρίζει ότι υπάρχει η mlab_foo(), αυτή δεν υπάρχει οντως στην glibc του target. Έτσι, ενώ προσθέσαμε την συνάρτηση στη glibc που χρησιμοποίησε ο cross-compiler, δεν θα μπορέσει να την βρει το εκτελέσιμο στο περιβάλλον του target.

C) Σε αυτή την περίπτωση, ο cross compiler θα φροντίσει να ενσωματώσει την συνάρτησή μας μέσα στο εκτελέσιμο, οπότε θα τρέξει στο target μηχανήμα, παρότι δεν διαθέτει το ίδιο την mlab_foo().

Άσκηση 2

1.

Default kernel:

```
uname -a
```

```
Linux debian-armhf 3.2.0-4-vexpress #1 SMP Debian 3.2.51-1  
armv7l GNU/Linux
```

Updated kernel:

```
uname -a
```

```
Linux debian-armhf 3.16.84 #3 SMP Tue Jan 6 17:54:52 EET 2026  
armv7l GNU/Linux
```

2.

Για την προσθήκη του custom system call κάναμε τα παρακάτω βήματα:

- Προσθέσαμε την εξής γραμμή στο τέλος του arch/arm/kernel/calls.S αρχείου,
`CALL(sys_custom_syscall)`
έτσι ώστε η CPU να γνωρίζει το ID του καινούριου system call.
- Προσθέσαμε ένα αρχείο στο kernel/ στο οποίο βρίσκεται η υλοποίηση του system call:

```
#include <linux/kernel.h>
```

```
#include <linux/syscalls.h>
```

```
SYSCALL_DEFINE0(custom_syscall)
```

```
{
```

```
    printk(KERN_INFO "Greetings from kernel and team no 20.\n");
```

```
    return 0;
```

```
}
```

- Προσθέσαμε το object code για την υλοποίηση του sys call στο Makefile, έτσι ώστε να γίνει build:

```
obj-y      = fork.o exec_domain.o panic.o \
            cpu.o exit.o itimer.o time.o softirq.o resource.o \
            sysctl.o sysctl_binary.o capability.o ptrace.o timer.o user.o \
            signal.o sys.o kmod.o workqueue.o pid.o task_work.o \
            extable.o params.o posix-timers.o \
            kthread.o sys_ni.o posix-cpu-timers.o \
            hrtimer.o nsproxy.o \
            notifier.o ksysfs.o cred.o reboot.o \
            async.o range.o groups.o smpboot.o custom_syscall.o
```

- Προσθέσαμε αυτή την γραμμή στο τέλος του αρχείου include/linux/syscalls.h :

```
asmlinkage long sys_custom_syscall(void);
```

για να δηλώσουμε στον compiler το syscall μας.

Μετά από αυτές τις αλλαγές, κάναμε build τον modified kernel.

3.

Το userspace πρόγραμμα που γράψαμε είναι το εξής:

```
#include <sys/syscall.h>
#include <stdio.h>

#define __NR_mysyscall 386
int main() {
    long res = syscall(__NR_mysyscall);
    if (res == 0) {
        printf("Syscall executed successfully!\n");
    }
    else {
        perror("Syscall failed");
    }
    return 0;
}
```


Εδώ, ο αριθμός 386 είναι το entry που έχει το syscall μας με βάση το αρχείο arch/arm/kernel/calls.S

Μεταγλωττίζοντας και τρέχοντας το παραπάνω πρόγραμμα, βλέπουμε ότι πράγματι, τυπώνεται το μήνυμα μας στα logs του πυρήνα:

```
root@debian-armhf ~# ./a.out
Syscall executed successfully!
root@debian-armhf ~# dmesg | tail
[ 24.925387] IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
[ 25.147445] RPC: Registered named UNIX socket transport module.
[ 25.150159] RPC: Registered udp transport module.
[ 25.152504] RPC: Registered tcp transport module.
[ 25.154754] RPC: Registered tcp NFSv4.1 backchannel transport module.
[ 25.202580] FS-Cache: Loaded
[ 25.323708] FS-Cache: Netfs 'nfs' registered for caching
[ 25.465255] Installing knfsd (copyright (C) 1996 okir@monad.swb.de).
[ 26.097785] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
[ 220.737682] Greetings from kernel and team no 20.
```